Natural Computing SoSe25

Exercise Sheet 7 — July 17th, 2025

Thomas Gabor, Maximilian Zorn, Karola Schneider

1 Evolutionary Algorithms

Consider the typical evolutionary algorithm discussed in the lecture and recall its most common parts:

typical evolutionary algorithm Let $\mathcal{E} = (\mathcal{X}, \mathcal{T}, \phi, E, \langle X_u \rangle_{0 \leq u \leq t})$ be a population-based optimization process as introduced in the lecture. [...]
The process $\mathcal{E}$ continues via a typical evolutionary algorithm if E has the form

$$\begin{aligned} E(\langle X_u \rangle_{0 \leq u \leq t}, \phi) = X_{t+1} = \sigma^{survivors}_{|X_t|}\big(X_t \\ \uplus\, mutate[\varrho^{mutants}_{M}(X_t)] \\ \uplus\, recombine[\sigma^{parents}_{2C}(X_t)] \\ \uplus\, migrate_H[\,]\big) \end{aligned}$$

where $M \in \mathbb{N}$ is the number of mutants produced per generation, $C \in \mathbb{N}$ is the number of children produced per generation, and $H \in \mathbb{N}$ is the number of migrants (sometimes called hypermutants) generated per generation. Especially M is often expressed as a rate for random choice within the population, i.e., $M = \lceil m \cdot |X_t| \rceil$ where $m \in \mathbb{R}$ is called mutation rate.

(i) In biology, the expression of genetic information is assigned various stages. Research and briefly explain the difference between the three concepts **genotype**, **phenotype**, and **fitness**.

(ii) Assume you are using an evolutionary algorithm to solve the following problems. In these scenarios, describe what your **genotype**, **phenotype**, **target function**, and **fitness function** are. How is the relation between your target function and your fitness function and why is it good or bad to use two separate functions here?

(a) You use an EA to find the fastest route for the Traveling Salesperson Problem. You want to visit towns $A, ..., H$. For each possible route, you use an online map service (and send JSON data) to compute the time the route takes. However, you noticed that the EA you use (and whose algorithm you cannot alter) tends to converge very early on local optima, which you want to avoid by using clever definitions.

(b) You want to find a pattern of size 100×100 in Conway's Game of Life that keeps generating new configurations of your 100×100 field of Game of Life for as long as possible.

(c) You use an EA to come up with the answers for an exam. To this end, you are searching the space of all strings but you use a spell checker to convert all words occurring in your arbitrary strings to the closest valid word in the English dictionary. To grade your exam, we assign your solution a number of *missing points* between 90 and 0. While your algorithm surely wants to minimize the missing points, your overall goal is to minimize the grade you receive (between 1.0 and 4.0).

(iii) Next, let's assume an evolutionary algorithm implemented in Python. The algorithm is optimizing individuals that are simply lists of four integers. Note that the Python function `random.random()` returns a random float between `0.0` and `1.0`. Consider the following snippet of Python code.

```
import random

def unknown_operator(individual_a, individual_b):
    new_individual = []
    for i, value_a in enumerate(individual_a):
       value_b = individual_b[i]
       if random.random() < 0.5:
           new_individual.append(value_a)
       else:
           new_individual.append(value_b)
    return new_individual

print(unknown_operator([0, 0, 0, 1], [1, 0, 0, 0]))
```

Give all possible unique print outputs of the above Python code snippet. State as precisely as possible which evolutionary operator is implemented here.
(Hint: It is not mutation)

(iv) We still consider an evolutionary algorithm optimizing individuals x that are lists of four integers, i.e., $x \in \mathbb{Z}^4$. In a suitable formalism (Python, pseudo-code, mathematics, ...) give a viable mutation function for these kinds of individuals. Said mutation function must be able to reach any arbitrary individual in the state space — when given any arbitrary individual as a starting point and infinite time — but should also not change the mutated individual completely.
(Hint: You can use the function `random.random()` as described in task (iii) and a function `round(`r`)`, which rounds float r to the nearest integer.)

2 Self-Adaptive Evolutionary Algorithms

Evolutionary algorithms (EAs) introduce many parameters, but some of them can be set self-adaptively, i.e., the evolutionary algorithm adjusts them automatically while it is running. Consider the following definition:

evolutionary algorithm with self-adaptive mutation rate
Let $\mathcal{E} = (\mathcal{X}, \mathcal{T}, \phi, E, \langle X_u \rangle_{0 \leq u \leq t})$ be the process of an evolutionary algorithm. $\mathcal{E}' = (\mathcal{X}', \mathcal{T}, \phi', E', \langle X'_u \rangle_{0 \leq u \leq t})$ is called an evolutionary algorithm with self-adaptive mutation rate if

- $\mathcal{X}' = \mathcal{X} \times \mathbb{R}$,
- $\phi'((x, m)) = \phi(x)$,
- a new meta-mutation weight $W \in \mathbb{R}$ is given alongside a function *rate-mutate* : $\mathbb{R} \to \mathbb{R}$ that has the form

  $$\textit{rate-mutate}(m) = \textit{cap}(m + W \cdot (r - 0.5))$$

  where *cap* maps all inputs beyond $[0; 1]$ to their nearest point within $[0; 1]$ (i.e., 0 or 1) and $r \sim [0; 1] \subseteq \mathbb{R}$ is a number drawn at random,
- E' uses a new mutation function *mutate*$'$ that has the form

  $$\textit{mutate}'((x, m)) = \begin{cases} (\textit{mutate}(x), \textit{rate-mutate}(m)) & \text{if } r \leq m, \\ (x, m) & \text{otherwise,} \end{cases}$$

  where *mutate* is the mutation function of E and $r \sim [0; 1] \subseteq \mathbb{R}$ is a number drawn at random,
- all individuals are subjected to the *mutate*$'$ function, i.e., $M = |X|$,
- E' uses a new recombination function *recombine*$'$ that has the form

  $$\textit{recombine}'((x_1, m_1), (x_2, m_2)) = \begin{cases} (\textit{recombine}(x_1, x_2), m_1) & \text{if } r \leq 0.5, \\ (\textit{recombine}(x_1, x_2), m_2) & \text{otherwise,} \end{cases}$$

  where *recombine* is the recombination function of E and $r \sim [0; 1] \subseteq \mathbb{R}$ is a number drawn at random,
- E' uses a new migration function *migrate*$'$ that has the form

  $$\textit{migrate}'() = (\textit{migrate}(), r)$$

  where *migrate* is the migration function of E and $r \sim [0; 1] \subseteq \mathbb{R}$ is a number drawn at random.

(i) In the evolutionary algorithm with self-adaptive mutation rate given above, there is no *selection pressure* on the mutation rate. Show which (part of a) definition ensures that! However, we should still expect a distinct trend in the values the individuals of $\mathcal{E}'$ have as mutation rate. Which trend and why does it occur? (Hint: Consider the likely fate of individuals that happen to have a high mutation rate.)

(ii) Can you give a reason why the behavior observed in task (i) above might be desirable, e.g., w.r.t. to the exploration/exploitation trade-off?

(iii) Now consider an **evolutionary algorithm with inherited recombination functions**. Here, every newly generated individual is given a personal recombination function $f = \textit{generate-recombine}()$ where *generate-recombine* is a randomized function that returns a randomly generated (but viable) recombination function f. When individuals are produced via recombination, one of their parents' recombination functions is chosen at random to perform the recombination on all the individual's data (except the assigned recombination function); for the assigned recombination function, the function used to produce the child is also assigned to the child. Once generated, the recombination functions are never changed. They do not affect an individual's fitness.
In line with the (lengthy) definition above, give a definition for an **evolutionary algorithm with inherited recombination functions**.

Let $\mathcal{E} = (\mathcal{X}, \mathcal{T}, \phi, E, \langle X_u \rangle_{0 \leq u \leq t})$ be the process of an evolutionary algorithm. $\mathcal{E}' = (\mathcal{X}', \mathcal{T}, \phi', E', \langle X'_u \rangle_{0 \leq u \leq t})$ is called an evolutionary algorithm with inherited recombination functions if ...

3 Creating Coconut Carriers

For this exercise we consider an evolutionary algorithm that we use to program a swallow bot, i.e., a robot of a small bird that we might use to carry coconuts, for example. For a little bit of biological plausibility, we encode the swallow bot's behavior via strings made up from 3000 letters A, C, G, and T, i.e., $\mathcal{X} = \{A, C, G, T\}^{3000}$.

(a) A nice one-point crossover function should recombine exactly one continuous part from each parent into one child. Naturally, all genes need to be at the correct place still. Furthermore, we assume that the recombination function receives the parents ordered by fitness, but any parent should have an equal chance of passing on any gene. **In a suitable formalism (Python, pseudo-code, mathematics, ...), give a nice one-point crossover function for the state space $\mathcal{X}$ as defined above.**
Note that you can use the Python function `random.random()`, which returns a random floating point number between `0.0` (inclusive) and `1.0` (exclusive), i.e., [0.0, 1.0), and you can use the notation $x \sim X$ to sample from a finite set X.

(b) We want our swallow bot to both have a high velocity and a high carrying capacity. Our swallow bot simulation provides us with both properties via two separate target functions

$$\tau_c : \mathcal{X} \to \mathbb{R}$$
$$\text{and } \tau_v : \mathcal{X} \to \mathbb{R}$$

so that, for a swallow bot with code $x \in \mathcal{X}$, $\tau_c(x)$ denotes its carrying capacity and $\tau_v(x)$ denotes its top velocity (when unladen). We have no way to meaningfully compare the values of both target functions with one another, but we can naturally compare values given by the same target function. Note that both τ_c and τ_v are to be maximized.

We now require a beautiful *multi-objective* selection function $\sigma_N^* : \wp^*(\mathcal{X}) \to \wp^*(\mathcal{X})$, which always returns exactly $N \in \mathbb{N}$ individuals from its input population X with $|X| \gg N$ and makes sure that both individuals with good τ_c and τ_v are potentially included. Furthermore, an individual that has the best target value in the population for both target functions at the same time should be guaranteed to be selected. **In a suitable formalism (Python, pseudo-code, mathematics, ...), give such a beautiful multi-objective selection function.**
Hint: You may want to build upon cut-off selection or tournament selection.